

Distributed Systems

Lecture5: Middleware

Mohammad Reza Shakournia

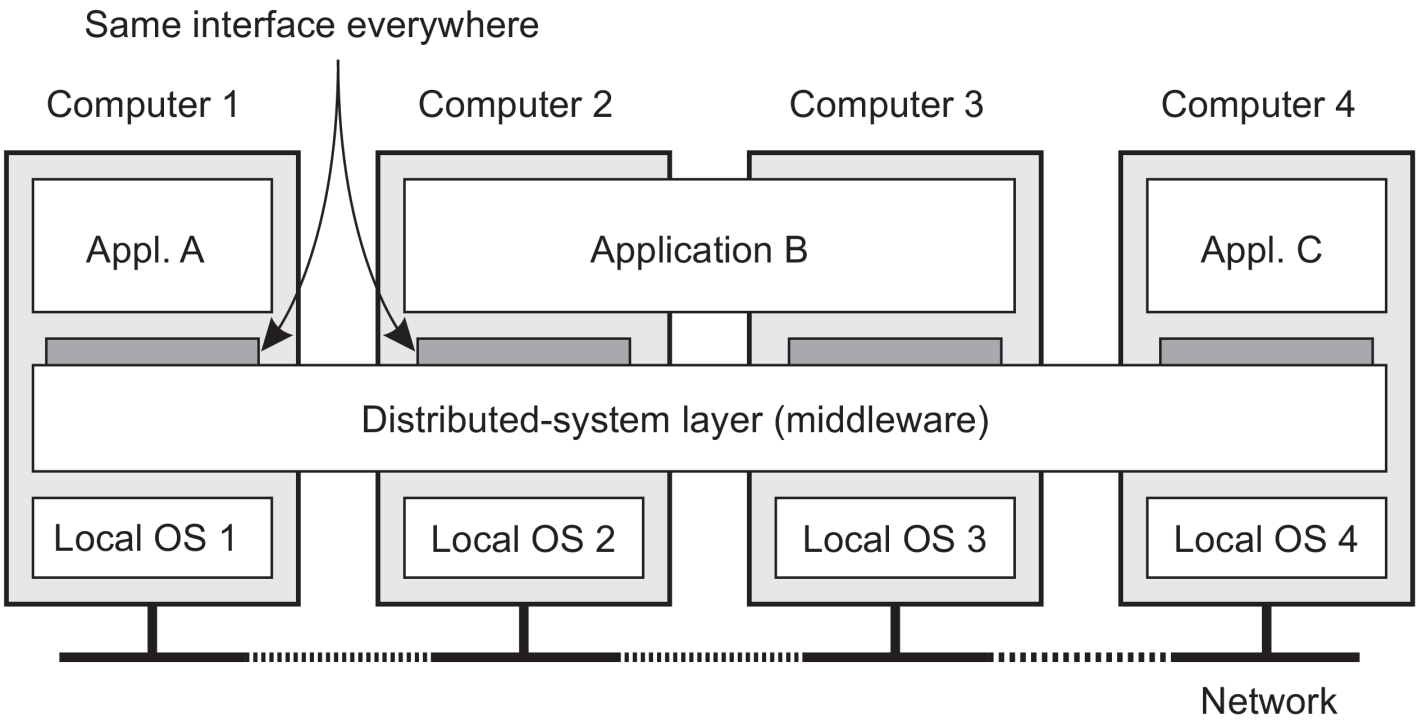
shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Middleware and Distributed Systems

- Distributed systems have a separate layer of software that is logically placed on top of the respective operating systems of each nodes.
- Distributed systems provide the means for inter application communications between different applications.
- At the same time, it hides the differences in hardware and operating systems from each application.
- Middleware is the same to a distributed system as what an operating system is to a computer; Responsible for:
  - Resource Management,
  - Facilities for interapplication communication,
  - Security services,
  - Accounting services,
  - Masking of and recovery from failures.

Middleware and Distributed Systems



Middleware Organization

- There are two important types of design patterns for the organization of middleware:
  - Wrappers,
  - Interceptors.
- Each targets different problems, yet addresses the same goal for middleware: **Achieving Openness.**
  - *See Lecture 2: An open distributed system is essentially a system that offers components that can easily be used by, or integrated into other systems.*

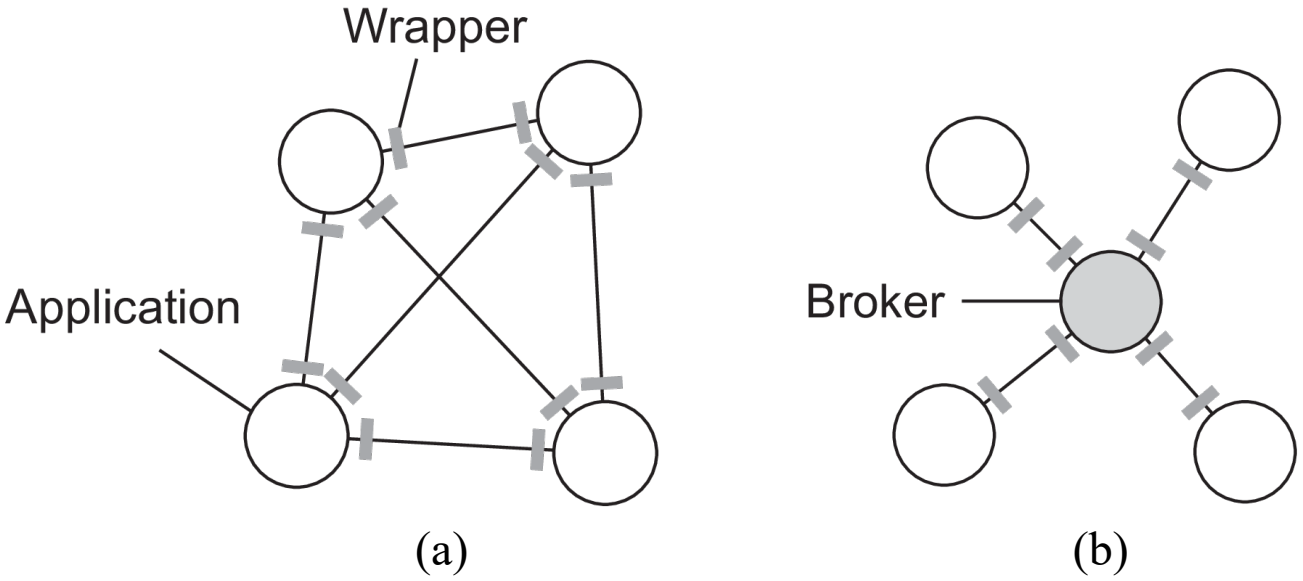
Wrappers

- Fundamental problem when building distributed systems:
  - The **interfaces** offered by the legacy component are most likely **not suitable for all applications.**
- We know that enterprise application integration could be established through middleware but in the end, components could be accessed through their native interface!
- Therefore, wrapper is here to help!

Definition 2.6. Wrapper

A wrapper or adapter is a special component that offers an interface acceptable to a client application, of which the functions are transformed into those available at the component.

Wrappers



(a) Requiring each application to have a wrapper for each other application. $\Longrightarrow \mathcal{O}(N^2)$

(b) Reducing the number of wrappers by making use of a broker. $\Longrightarrow \mathcal{O}(N)$

Interceptors

Definition 2.7. Interceptors

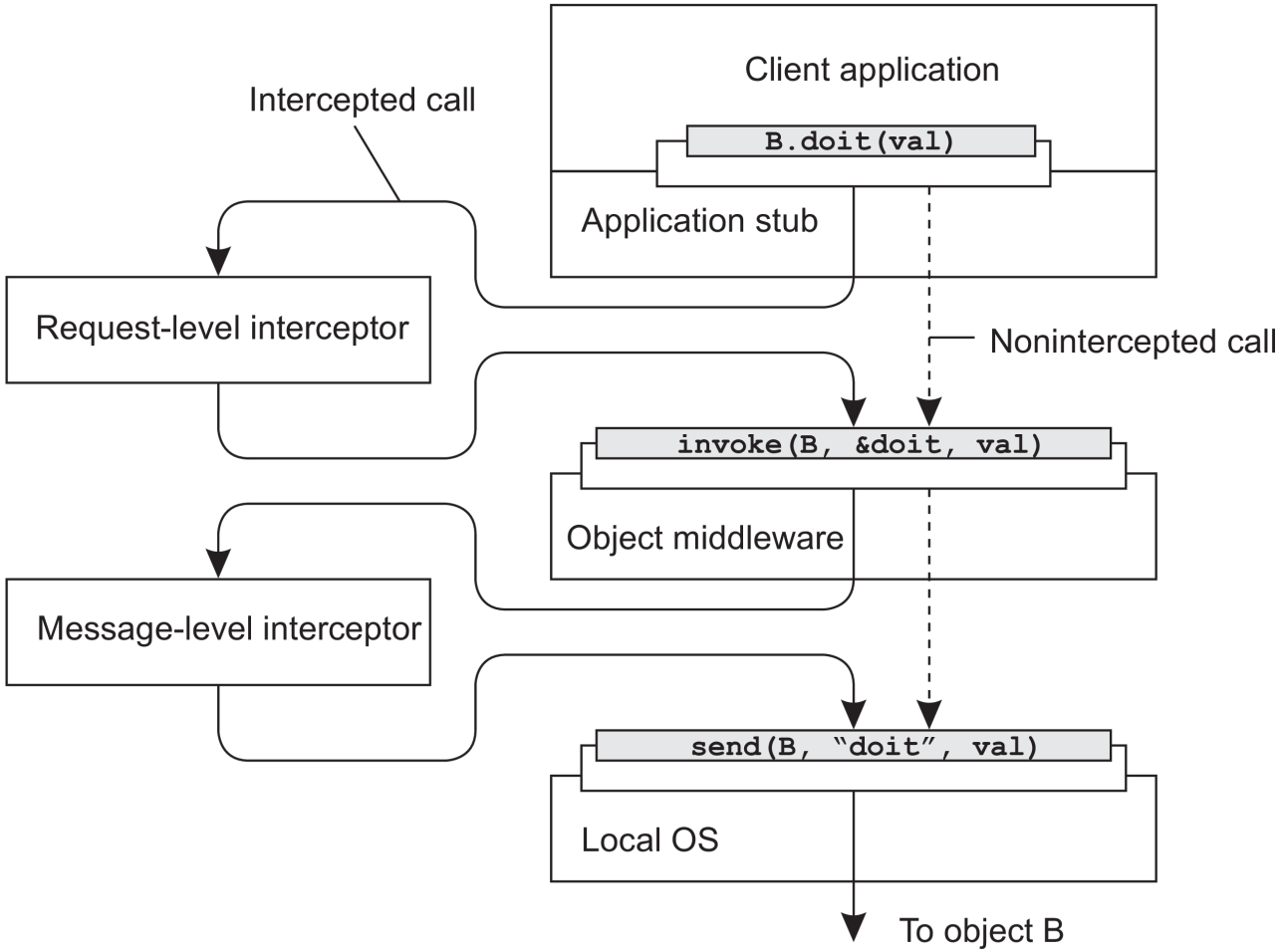
An interceptor is nothing but a software construct that will break the usual flow of control and allow other (application specific) code to be executed.

- Interceptors play an important role in making middleware open.
- The basic idea is simple:
  - an object A can call a method that belongs to an object B, while the latter resides on a different machine than A.
- For example, remote-object invocations could be handled using interceptors.

Interceptors

- **Remote-object invocation** is carried out in three steps.
  1. Object A is offered a local interface that is the same as the interface offered by object B. A calls the method available in that interface.
  2. The call by A is transformed into a generic object invocation, made possible through a general object-invocation interface offered by the middleware at the machine where A resides.
  3. Finally, the generic object invocation is transformed into a message that is sent through the transport-level network interface as offered by A's local operating system.

Interceptors



Modifiable/Adaptive Middleware

- Middleware is responsible for reacting to the continuous changes of the distributed systems.
- Changes include those resulting from: mobility, a strong variance in the quality-of service of networks, failing hardware, and battery drainage, etc.
- Moreover, as the size of a distributed system increases, changing its parts can rarely be done by temporarily shutting it down.
- **What is needed is being able to make changes on-the-fly.**
- Interceptors can be a means to adapt the standard flow of control.
- Replacing software components at runtime is an example of modifying a system.
- The most popular approaches toward modifiable middleware is that of dynamically constructing middleware from components.

Modifiable Middleware

- A system may either be configured:
  - statically at design time,
  - or dynamically at runtime (late binding).
- Late binding is a technique operating systems where modules can be loaded and unloaded at will. (Complex for DS)
- To accommodate dynamic changes to the software that makes up middleware, we need at least basic support to:
  - **load and unload components at runtime**.
  - for each component explicit specifications of the interfaces it offers, as well the interfaces it requires, are needed.

Wrap-Up + What's Next

In the next lecture, we will discuss layered-systems architectures such as:

- Client-server architecture,
- Multitiered Architectures.